

NoSQL

Gestión y Arquitectura de Datos



¿Por qué surge NoSQL?

Las bases de datos NoSQL surgieron como respuesta a limitaciones específicas de las bases de datos relacionales:

- **Escalabilidad:** Escalado horizontal vs vertical
- **Flexibilidad:** Esquemas rígidos vs flexibles
- **Big Data:** Manejo de datos no estructurados
- **Velocidad:** Alta velocidad en lectura/escritura
- **Disponibilidad:** Sistemas distribuidos

Tipos de Bases de Datos NoSQL



Documentales



Clave-Valor



Columnares



De Grafos

Características:

- Almacenan datos en documentos JSON
- Flexibilidad en esquemas
- Datos anidados y no estructurados
- Fáciles de usar



Ejemplo de Base de Datos Documental

```
1 {  
2   "id": "1",  
3   "cliente": {  
4     "nombre": "Juan Garcia",  
5     "email": ["juan@email.com", "juan.g@email.com"],  
6     "direcciones": [  
7       {  
8         "tipo": "casa",  
9         "calle": "Av. Libertador 1234"  
10      }  
11    ]  
12  },  
13  "pedidos": [  
14    {  
15      "fecha": "2024-01-01",  
16      "total": 1500  
17    }  
18  ]  
19 }
```

Características:

- Almacenan pares clave-valor
- Alta velocidad
- Simplicidad
- Ideales para cachés



Ejemplo de Base de Datos Clave-Valor

```
1 SET usuario:1 "Juan Garcia"  
2 SET usuario:1:email "juan@email.com"  
3 SET carrito:1 "['producto1', 'producto2']"
```

Características:

- Almacenan datos en columnas
- Eficiencia en análisis
- Escalabilidad horizontal
- Mejor compresión

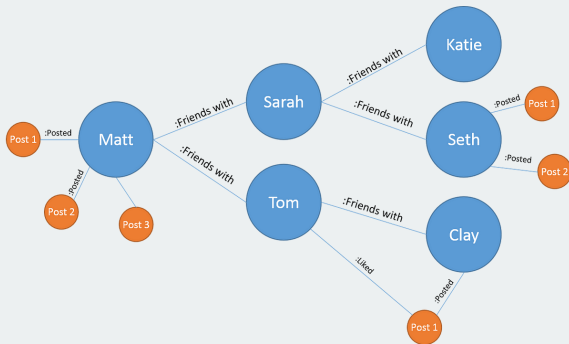


Características:

- Representan relaciones entre entidades
- Consultas eficientes de relaciones
- Ideales para redes sociales
- Sistemas de recomendación



Ejemplo de Grafo



Un sistema distribuido no puede garantizar simultáneamente:



Consistency

Todos ven los
mismos datos



Availability

Cada petición
recibe respuesta



Partition Tolerance

Funciona aún con fallos
de red

Clasificación CAP de Bases de Datos

Base de Datos	Prioriza	Sacrifica
MongoDB	CP	Availability
Cassandra	AP	Consistency
Redis	AP	Consistency
Neo4j	CA	Partition Tolerance

Nota: MongoDB es configurable para priorizar Consistency o Availability. Redis por defecto prioriza AP (replicación asíncrona), pero puede configurarse como CP.

Casos de uso ideales:

- **Datos no estructurados:** Sin esquema fijo
- **Escalabilidad horizontal:** Añadir servidores
- **Alta velocidad:** Lectura/escritura rápidas
- **Agilidad:** Desarrollo rápido
- **Big Data:** Grandes volúmenes de datos

Cuándo NO Usar NoSQL

Casos donde evitar NoSQL:

- **Datos altamente relacionales:** Muchas relaciones complejas
- **Transacciones ACID:** Garantías estrictas
- **Reportes complejos:** Múltiples JOINS
- **Equipos sin experiencia:** Curva de aprendizaje

Comparación: SQL vs NoSQL

Característica	SQL	NoSQL
Esquema	Fijo	Flexible
Escalabilidad	Vertical	Horizontal
Transacciones	ACID	Eventual
Consultas	Complejas	Simples
Curva aprendizaje	Baja	Media-Alta

¿Dudas?
¿Consultas?



Universidad de
SanAndrés